

GadgetXpress

Reporte de Refactorizacion — C++

Principio de Responsabilidad Unica y Diseno Modular

Proyecto:	Sistema de Inventario GadgetXpress
Lenguaje:	C++ (estandar ISO 11/14)
Enfoque:	Refactorizacion — Principio de Responsabilidad Unica (SRP)
Objetivo:	Transformar codigo funcional en codigo mantenible y escalable

1. Contexto del Proyecto

GadgetXpress enfrenta desafios en la gestion manual de su inventario. En las evidencias previas se construyo en C++ un sistema funcional que permitia registrar productos, actualizar cantidades y generar reportes. Sin embargo, el codigo mezcla multiples responsabilidades en pocas funciones, lo que dificulta su mantenimiento y escalabilidad.

Objetivo de esta evidencia: Refactorizar el codigo aplicando el Principio de Responsabilidad Unica (SRP) para que cada funcion realice una sola tarea bien definida, preparando el sistema para futuras funcionalidades como la simulacion de ventas.

2. Analisis del Codigo Original

Problemas identificados

#	Problema	Impacto
1	El main mezcla UI, logica y control de flujo	Dificil de leer y modificar
2	No existe funcion para agregar ni listar productos	Funcionalidad incompleta
3	La busqueda de productos se duplicaria en cada funcion	Codigo redundante (DRY violado)
4	Uso de arreglos de tamano fijo (MAX=10)	No escala con el inventario real
5	Caracteres mal codificados (Ã³, Ã¡)	Ilegibilidad en distintos sistemas
6	Sin validacion de entradas del usuario	Crashes ante datos incorrectos

3. Antes y Despues — Funcion main

ANTES — main original (mezcla de responsabilidades)

El main original contenia directamente: la inicializacion de datos, el ciclo do-while, la impresion del menu Y la estructura switch — todo en un mismo bloque sin separacion de capas.

```
int main() {
    const int MAX = 10;
    Producto inventario[MAX];
    int numProductos = 4;

    inventario[0] = {"Laptop Gamer", "LAP-001", 12, 3500.00};
    // ... mas datos precargados ...

    int opcion;
    do {
        // Menu impreso DIRECTAMENTE en main
        cout << "GADGETXPRESS - INVENTARIO\n";
        cout << "1. Actualizar cantidad\n";
        cout << "2. Generar reporte bajo inventario\n";
        cout << "3. Salir\n";
        cout << "Opcion: ";
        cin >> opcion;

        switch(opcion) {
            case 1: actualizarCantidad(inventario, numProductos); break;
            case 2: generarReporte(inventario, numProductos); break;
            case 3: cout << "Saliendo...\n"; break;
            default: cout << "Opcion invalida.\n";
        }
    } while (opcion != 3);
    return 0;
}
```

DESPUES — main refactorizado (solo orquesta)

El nuevo main DELEGA todo: la impresion del menu a mostrarMenu(), y cada accion a su funcion especializada. El main solo contiene el bucle principal y el switch de despacho.

```
int main() {
    // Solo inicializacion de datos de prueba
    vector inventario = {
```

```

{"Laptop Gamer", "LAP-001", 12, 3500.00},
{"Mouse USB", "MOU-002", 3, 45.50},
{"Teclado RGB", "TEC-003", 8, 120.00},
{"Monitor 24", "MON-004", 2, 850.00}
};

int opcion;
do {
    mostrarMenu(); // <-- Delega impresion del menu
    cin >> opcion;

    switch (opcion) {
        case 1: agregarProducto(inventario); break;
        case 2: listarProductos(inventario); break;
        case 3: actualizarCantidad(inventario); break;
        case 4: generarReporte(inventario); break;
        case 5: cout << "Hasta luego!\n"; break;
        default: cout << "Opcion invalida.\n";
    }
} while (opcion != 5);

return 0; // Unica responsabilidad: orquestar el flujo
}

```

4. Funciones Implementadas y su Responsabilidad

Funcion	Responsabilidad Unica	Tipo
mostrarMenu()	Imprimir opciones del menu en pantalla	UI
agregarProducto(inventario)	Solicitar datos y agregar un nuevo producto	Negocio
listarProductos(inventario)	Mostrar todos los productos en tabla	UI / Consulta
actualizarCantidad(inventario)	Buscar producto y modificar su cantidad	Negocio
generarReporte(inventario)	Filtrar y mostrar productos con stock < 5	Reporte
buscarProducto(inventario, codigo)	Retornar el indice de un producto por codigo	Ayuda
leerEnteroPositivo(mensaje)	Validar y leer un entero no negativo del usuario	Ayuda
leerDoublePositivo(mensaje)	Validar y leer un decimal positivo del usuario	Ayuda

5. Funcion de Ayuda: buscarProducto

Se identifico que la logica de busqueda por codigo se repetiria en al menos dos lugares: al actualizar la cantidad y al agregar un producto (para evitar duplicados). Siguiendo el principio DRY (Don't Repeat Yourself), se extrajo a una

funcion de ayuda independiente:

```
// Retorna el indice del producto o -1 si no existe
int buscarProducto(const vector& inventario, const string& codigo) {
    for (int i = 0; i < (int)inventario.size(); ++i) {
        if (inventario[i].codigo == codigo)
            return i;
    }
    return -1;
}
```

Llamadas desde otras funciones:

```
// En agregarProducto – evita codigos duplicados:
if (buscarProducto(inventario, nuevo.codigo) != -1) { ... }

// En actualizarCantidad – localiza el producto a modificar:
int idx = buscarProducto(inventario, cod);
if (idx == -1) { cout << "Codigo no encontrado"; return; }
inventario[idx].cantidad = nuevaCant;
```

6. Mejoras Adicionales Aplicadas

Mejora	Descripcion
vector<Producto>	Se reemplazo el arreglo estatico (MAX=10) por vector dinamico, eliminando el limite arbitrario de productos
Funciones agregar y listar	El codigo base no tenia estas funciones. Se agregaron cumpliendo el requisito minimo del taller.
Validacion de entradas	Las funciones leerEnteroPositivo() y leerDoublePositivo() previenen crashes por datos invalidos.
Codificacion correcta	Se corrigieron los caracteres mal codificados (Ã³ -> o, etc.) para portabilidad en distintos SO.
Documentacion en codigo	Cada funcion incluye un comentario de bloque que describe su responsabilidad unica.

7. Beneficios de la Nueva Estructura

Mantenibilidad	Al tener responsabilidades separadas, modificar una funcion no afecta a las demas. Si el cliente cambia el umbral del reporte, solo se toca generarReporte().
Escalabilidad	Agregar nuevas funcionalidades (ventas, exportar CSV) solo requiere crear nuevas funciones y un case en el switch — sin tocar el codigo existente.
Legibilidad	El main de 5 lineas utiles comunica el flujo del programa de un vistazo. Cualquier desarrollador nuevo entiende la arquitectura en segundos.

Reutilizacion	buscarProducto() se llama desde multiples lugares sin duplicar logica, reduciendo el riesgo de bugs por inconsistencias.
Testabilidad	Funciones pequenas y enfocadas son mas faciles de probar de forma aislada, preparando el codigo para pruebas unitarias en el futuro.

8. Conclusion

La refactorizacion del sistema GadgetXpress demostro que un codigo funcional puede — y debe — evolucionar hacia un codigo de calidad profesional. Aplicando el Principio de Responsabilidad Unica, pasamos de un programa donde el main concentraba toda la logica, a una arquitectura modular con 9 funciones claramente definidas.

El resultado es un sistema mantenible, escalable y listo para recibir nuevas funcionalidades como la simulacion de ventas, sin necesidad de reescribir el codigo existente. Esta es la diferencia entre software que simplemente funciona y software de alta calidad.

Principio aplicado: "Una funcion debe tener una, y solo una, razon para cambiar." — Robert C. Martin (Uncle Bob), Clean Code.